

Chapter 3 & 4 : TARGET LABORATORY SOURCE CODE & LABORATORY PROCEDURES AND TASKS.

```
root@Ubuntu:~/biraj/2ndir/lab4.2# gcc -g code1.c -o code1
root@Ubuntu:~/biraj/2ndir/lab4.2# gdb ./code1
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./code1...
```

Figure: Compilation using gdb debugger

```
(gdb) break 22
Breakpoint 1 at 0x1241: file code1.c, line 22.
(gdb) run
Starting program: /root/biraj/2ndir/lab4.2/code1

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Addresses:
BSS (Global): 0x55555555014

Breakpoint 1, main () at code1.c:22
22      printf("Stack (Local): %p\n", (void *)&stack_local_var);
(gdb) info files
Symbols from "/root/biraj/2ndir/lab4.2/code1".
Native process:
  Using the running image of child process 5278.
  While running this, GDB does not access memory from...
Local exec file:
  '/root/biraj/2ndir/lab4.2/code1', file type elf64-x86-64.
  Entry point: 0x555555550e0
  0x0000555555554350 - 0x00005555555554374 is .note.gnu.build-id
  0x0000555555554374 - 0x00005555555554390 is .interp
  0x0000555555554390 - 0x000055555555543b4 is .gnu.hash
  0x00005555555543b8 - 0x000055555555544c0 is .dynsym
  0x00005555555544c0 - 0x0000555555555457b is .dynstr
```

Figure: Use of gdb commands

Summary: The breakpoint pauses the program before the final `printf()` statement, allowing the BSS, stack, and heap memory regions to be inspected while the program is still running. At this point, the dynamically allocated heap memory and local stack variable are still valid.

Chapter 4 task E continue:

```
(gdb) print &bss_global_var
$1 = (int *) 0x555555558014 <bss_global_var>
(gdb) info proc mappings
process 5278
Mapped address spaces:
```

Start Addr	End Addr	Size	Offset	Perms	File
0x000055555554000	0x000055555555000	0x1000	0x0	r--p	/root/biraj/2ndir/lab4.2/code1
0x000055555555000	0x000055555556000	0x1000	0x1000	r-xp	/root/biraj/2ndir/lab4.2/code1
0x000055555556000	0x000055555557000	0x1000	0x2000	r--p	/root/biraj/2ndir/lab4.2/code1
0x000055555557000	0x000055555558000	0x1000	0x2000	r--p	/root/biraj/2ndir/lab4.2/code1
0x000055555558000	0x000055555559000	0x1000	0x3000	rw-p	/root/biraj/2ndir/lab4.2/code1
0x000055555559000	0x00005555555a000	0x21000	0x0	rw-p	[heap]
0x00007ffff7c0000	0x00007ffff7c28000	0x28000	0x0	r--p	/usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7c28000	0x00007ffff7dc0000	0x198000	0x28000	r-xp	/usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7dc0000	0x00007ffff7e0e000	0x4e000	0x1c0000	r--p	/usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e0e000	0x00007ffff7e12000	0x4000	0x20e000	r--p	/usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e12000	0x00007ffff7e14000	0x2000	0x212000	rw-p	/usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e14000	0x00007ffff7e21000	0xd000	0x0	rw-p	
0x00007ffff7fa3000	0x00007ffff7fa6000	0x3000	0x0	rw-p	
0x00007ffff7fb6000	0x00007ffff7fb8000	0x2000	0x0	rw-p	
0x00007ffff7fb8000	0x00007ffff7fbc000	0x4000	0x0	r--p	[vvar]
0x00007ffff7fbc000	0x00007ffff7fbe000	0x2000	0x0	r--p	[vvar_vclock]
0x00007ffff7fbe000	0x00007ffff7fc0000	0x2000	0x0	r-xp	[vdso]
0x00007ffff7fc0000	0x00007ffff7fc1000	0x1000	0x0	r--p	/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.s
o.2					
0x00007ffff7fc1000	0x00007ffff7ff0000	0x2f000	0x1000	r-xp	/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.s
o.2					
0x00007ffff7ff0000	0x00007ffff7ffb000	0xb000	0x30000	r--p	/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.s
o.2					
0x00007ffff7ffb000	0x00007ffff7ffd000	0x2000	0x3b000	r--p	/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.s
o.2					
0x00007ffff7ffd000	0x00007ffff7ffe000	0x1000	0x3d000	rw-p	/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.s
o.2					
0x00007ffff7ffe000	0x00007ffff7fff000	0x1000	0x0	rw-p	
0x00007ffff7fff000	0x00007ffff7fff000	0x21000	0x0	rw-p	[stack]
0xfffffffff60000	0xfffffffff601000	0x1000	0x0	--xp	[vsyscall]

```
(gdb) print stack_local_var
$2 = 42
(gdb) print &stack_local_var
$3 = (int *) 0x7fffffe1bc
(gdb) x/42x $sp
0x7fffffe1b0: 0x00000000 0x00000000 0x00000000 0x0000002a
0x7fffffe1c0: 0x55559010 0x00005555 0x49282500 0x607ef6f5
0x7fffffe1d0: 0xfffffe280 0x00007fff 0xf7c2a601 0x00007fff
0x7fffffe1e0: 0xfffffe2c0 0x00007fff 0xfffffe308 0x00007fff
```

Figure: Extracting OS Kernel Address Tables.

Summary: The heap contains dynamically allocated memory created using malloc(), while the stack contains local variables, function call information, and other temporary execution data.

```

(gdb) print stack_local_var
$2 = 42
(gdb) print &stack_local_var
$3 = (int *) 0x7fffffff1bc
(gdb) x/42x $sp
0x7fffffff1b0: 0x00000000    0x00000000    0x00000000    0x0000002a
0x7fffffff1c0: 0x55559010    0x00005555    0x49282500    0x607ef6f5
0x7fffffff1d0: 0xffffe280    0x00007fff    0xf7c2a601    0x00007fff
0x7fffffff1e0: 0xffffe2c0    0x00007fff    0xffffe308    0x00007fff
(gdb) x/42x $sp
0x7fffffff1b0: 0x00000000    0x00000000    0x00000000    0x0000002a
0x7fffffff1c0: 0x55559010    0x00005555    0x49282500    0x607ef6f5
0x7fffffff1d0: 0xffffe280    0x00007fff    0xf7c2a601    0x00007fff
0x7fffffff1e0: 0xffffe2c0    0x00007fff    0xffffe308    0x00007fff
0x7fffffff1f0: 0xf7fc0000    0x00000001    0x555551c9    0x00005555
0x7fffffff200: 0xffffe240    0x00007fff    0xf7fe2dc6    0x00007fff
0x7fffffff210: 0x00000000    0x00000000    0x98f3185e    0x405b5e56
0x7fffffff220: 0xffffe308    0x00007fff    0x00000001    0x00000000
0x7fffffff230: 0xf7ffd000    0x00007fff    0x55557da0    0x00005555
0x7fffffff240: 0x9e33185e    0x405b5e56    0x1699185e    0x405b4e2c
0x7fffffff250: 0x00000000    0x00007fff
(gdb)

```

Figure: Examining the active variables mapped inside the local stack workspace

```

0x7fffffff230: 0x00000000    0x00007fff
(gdb) print heap_ptr
$4 = (int *) 0x55555559010
(gdb) x/lwd heap_ptr
0x55555559010: 99
(gdb)

```

Figure: Evaluating the literal dynamic storage state

```

0x55555559010: 99
(gdb) continue
Continuing.
Stack (Local): 0x7fffffff1bc
Heap (Dynamic): 0x55555559010
Ready for GDB inspection!
[Inferior 1 (process 5278) exited normally]
(gdb) quit
root@Ubuntu:~/biraj/2ndir/lab4.2#

```

Figure: Terminating the program cleanly and detaching tracking engines.

Summary: Task F focuses on examining the stack and heap memory using GDB. The local variable `stack_local_var` is inspected to verify its value as 42 and to find its stack address. The command `x/4wx $sp` displays raw memory values near the current stack pointer in hexadecimal. The `heap_ptr` is then examined to find the address of the dynamically allocated heap memory. Using `x/1wd heap_ptr`, the stored heap value 99 is displayed in decimal. Finally, `continue` resumes the program and exits GDB.